



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO **“Alineación Futbol”**

Curso: Programación II

Docente: Breno Solim Luque Zuñiga

Jiménez Romero, Josué Andre (2022074259)

Tacna – Perú
2026



INDICE

Contenido

INDICE.....	2
I. INFORMACIÓN GENERAL	3
- Objetivos	3
- Equipos, materiales, programas y recursos utilizados.....	3
II. MARCO TEORICO.....	3
III. PROCEDIMIENTO.....	4
IV. ANALISIS E INTERPRETACION DE RESULTADOS.....	14
CONCLUSIONES	14
RECOMENDACIONES.....	14



INFORME DE LABORATORIO

TEMA: Alineación Futbol

I. INFORMACIÓN GENERAL

- **Objetivos:** Aplicar los conceptos fundamentales de la Programación Orientada a Objetos (POO), el manejo de eventos, la manipulación de colecciones dinámicas en memoria (CRUD) y la renderización de gráficos 2D en C# a través del desarrollo de una aplicación de escritorio (Windows Forms) para gestionar táctica y visualmente un equipo de fútbol.
- **Equipos, materiales, programas y recursos utilizados:**
 - Azure Data Studio
 - Base de datos: TSQL.bak
 - Al menos 4GB de RAM.
 - Librerías gráficas: System.Collections.Generic, System.Linq, System.Drawing, System.Windows.Forms.

II. MARCO TEORICO

El laboratorio se estructuró en cuatro partes que reflejan la arquitectura básica de un videojuego 2D:

- **Parte I:** Programación Orientada a Objetos (POO) y Encapsulamiento
- Se creó la clase entidad Jugador para encapsular los atributos principales (Nombre, Posición, Número de camiseta y Condición). Esto permite instanciar objetos independientes, separando los datos puros de la interfaz gráfica y facilitando el mantenimiento del código.
- **Parte II: Colecciones en Memoria y Operaciones CRUD**

Para cumplir con la restricción de no usar bases de datos, se implementó una colección genérica List<Jugador> como motor de almacenamiento. Se desarrollaron



algoritmos enlazados a eventos de botones para ejecutar operaciones de creación (Agregar), lectura/actualización en componentes DataGridView (Modificar) y eliminación por índice (Eliminar).

- **Parte III: Reglas de Negocio y Consultas con LINQ**

Se incorporó lógica de validación para mantener la coherencia del dominio del problema (ej. máximo 11 jugadores titulares). Se utilizaron expresiones lambda y métodos de extensión de LINQ (Where, Count) para consultar la colección en tiempo real antes de permitir modificaciones de estado.

- **Parte IV: Gráficos y Renderizado Dinámico (GDI+)**

Se utilizó el evento Paint de un formulario secundario (FormCancha) junto con la clase Graphics para dibujar la distribución táctica. Mediante algoritmos matemáticos, se calculó el espaciado dinámico basado en la cantidad de jugadores por línea, renderizando elipses (FillEllipse) y textos (DrawString) para representar gráficamente a los titulares sobre el campo de juego.

III. PROCEDIMIENTO

Control	Propiedad	Valor
Form1	Text, BackColor, Font, StartPosition, Size	"Alineación de Fútbol", White, Segoe UI 9.5, CenterScreen, 680; 587
lblNombre	Text, Font, ForeColor	"Nombre del Jugador", Segoe UI 9.5, RGB(64, 64, 64)
txtNombre	BorderStyle, Font	FixedSingle, Segoe UI 10



INFORME DE LABORATORIO– Alineacion de futbol

lblPosicion	Text, Font, ForeColor	"Posición", Segoe UI 9.5, RGB(64, 64, 64)
cmbPosicion	DropDownStyle, Font, Items	DropDownList, Segoe UI 10, "Portero", "Defensa", "Mediocampista", "Delantero"
lblNumero	Text, Font, ForeColor	"Número de Camiseta", Segoe UI 9.5, RGB(64, 64, 64)
numCamiseta	BorderStyle, Font, Minimum, Maximum, Value	FixedSingle, Segoe UI 10, 1, 99, 1
lblCondicion	Text, Font, ForeColor	"Condición", Segoe UI 9.5, RGB(64, 64, 64)
cmbCondicion	DropDownStyle, Font, Items	DropDownList, Segoe UI 10, "Titular", "Suplente"
btnAgregar	Text, BackColor, ForeColor, Font, FlatStyle	"Agregar", RGB(0, 122, 255), White, Segoe UI 9.5 Semibold, Flat
btnModificar	Text, BackColor, ForeColor, Font, FlatStyle	"Modificar", RGB(229, 229, 234), Black, Segoe UI 9.5 Semibold, Flat
btnEliminar	Text, BackColor, ForeColor, Font, FlatStyle	"Eliminar", RGB(255, 59, 48), White, Segoe UI 9.5 Semibold, Flat
btnMT	Text, BackColor, ForeColor, Font, FlatStyle	"Mostrar titulares", RGB(0, 122, 255), White, Segoe UI 9.5 Semibold, Flat
dgvJugadores	BackgroundColor, AutoSizeColumnsMode, SelectionMode, ReadOnly, AllowUserToAddRows	White, Fill, FullRowSelect, True, False



INFORME DE LABORATORIO– Alineacion de futbol

Figura 1

Modelo del formulario Alineacion_Futbol

Alineación de Fútbol

Nombre del Jugador

Carlos Zambrano

Agregar

Posición

Defensa

Modificar

Número de Camiseta

5

Eliminar

Condición

Titular

Mostrar titulares

Nombre	Posicion	NumeroCamiseta	Condicion
Pedro Gallese	Portero	1	Titular
Luis Advíncula	Defensa	17	Titular
Carlos Zambrano	Defensa	5	Titular
Alexander Callens	Defensa	22	Titular
Marcos López	Defensa	4	Titular
Renato Tapia	Mediocampista	13	Titular

Figura 2

Diseño de formulario cancha



INFORME DE LABORATORIO– Alineacion de futbol



Figura 3

Creación de clase Jugador

```
namespace AlineacionFutbol
{
    public class Jugador
    {
        public string Nombre { get; set; }
        public string Posicion { get; set; }
        public int NumeroCamiseta { get; set; }
        public string Condicion { get; set; }
    }
}
```



Figura 4

Botón agregar

```
1 referencia
private void btnAgregar_Click(object sender, EventArgs e)
{
    //Validamos que los campos no estén vacíos
    if (string.IsNullOrEmpty(txtNombre.Text) ||
        cmbPosicion.SelectedIndex == -1 ||
        cmbCondicion.SelectedIndex == -1)
    {
        MessageBox.Show("Por favor, completa todos los campos antes de agregar.", "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    //Validacion solo 11 titulares
    string condicionElegida = cmbCondicion.SelectedItem.ToString();
    if (condicionElegida == "Titular")
    {
        //Contamos cuántos titulares hay actualmente en la lista
        int cantidadTitulares = listaJugadores.Count(j => j.Condicion == "Titular");

        if (cantidadTitulares >= 11)
        {
            MessageBox.Show("El equipo ya cuenta con 11 jugadores titulares. Debe registrar al jugador como Suplente.", "Límite alcanzado",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }
    }

    Jugador nuevoJugador = new Jugador();
    nuevoJugador.Nombre = txtNombre.Text;
    nuevoJugador.Posicion = cmbPosicion.SelectedItem.ToString();
    nuevoJugador.NumeroCamiseta = (int)numCamiseta.Value;
    nuevoJugador.Condicion = condicionElegida;

    listaJugadores.Add(nuevoJugador);
    ActualizarGrid();
    LimpiarCampos();
}
}
```

Figura 5

Botón modificar

```
1 referencia
private void btnModificar_Click(object sender, EventArgs e)
{
    //Verificamos que haya una fila seleccionada
    if (dgvJugadores.CurrentRow == null)
    {
        MessageBox.Show("Por favor, selecciona un jugador de la tabla para modificar.", "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    //Validamos que los campos no estén vacíos
    if (string.IsNullOrEmpty(txtNombre.Text) ||
        cmbPosicion.SelectedIndex == -1 ||
        cmbCondicion.SelectedIndex == -1)
    {
        MessageBox.Show("Por favor, completa todos los campos antes de modificar.", "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Warning);
        return;
    }

    int indiceSeleccionado = dgvJugadores.CurrentRow.Index;
    string nuevaCondicion = cmbCondicion.SelectedItem.ToString();

    //Validacion limite de 11 jugadores titulares maximo
    if (nuevaCondicion == "Titular")
    {
        int cantidadTitulares = listaJugadores.Where((j, index) => index != indiceSeleccionado && j.Condicion == "Titular").Count();

        if (cantidadTitulares >= 11)
        {
            MessageBox.Show("Ya existen 11 jugadores titulares en el equipo. No puedes cambiar a este jugador a Titular.", "Límite alcanzado",
                MessageBoxButtons.OK, MessageBoxIcon.Warning);
            return;
        }
    }

    listaJugadores[indiceSeleccionado].Nombre = txtNombre.Text;
    listaJugadores[indiceSeleccionado].Posicion = cmbPosicion.SelectedItem.ToString();
    listaJugadores[indiceSeleccionado].NumeroCamiseta = (int)numCamiseta.Value;
    listaJugadores[indiceSeleccionado].Condicion = nuevaCondicion;

    ActualizarGrid(); //refrescar GRID despues de modificar
    LimpiarCampos();
    MessageBox.Show("Jugador modificado correctamente.", "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
}
```




Figura 6

Botón eliminar

```
1 referencia
private void btnEliminar_Click(object sender, EventArgs e)
{
    if (dgvJugadores.CurrentRow == null)
    {
        MessageBox.Show("Por favor, selecciona un jugador de la tabla para eliminar.", "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    DialogResult respuesta = MessageBox.Show("¿Estás seguro de que deseas eliminar a este jugador?", "Confirmar Eliminación", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

    if (respuesta == DialogResult.Yes)
    {
        int indiceSeleccionado = dgvJugadores.CurrentRow.Index;
        listaJugadores.RemoveAt(indiceSeleccionado);

        ActualizarGrid();
        LimpiarCampos();

        MessageBox.Show("Jugador eliminado correctamente.", "Éxito", MessageBoxButtons.OK, MessageBoxIcon.Information);
    }
}
```

Figura 7

Asignar la lista de jugadores e instanciar en el formulario Cancha

```
private void btnMT_Click(object sender, EventArgs e)
{
    List<Jugador> titulares = listaJugadores.Where(j => j.Condicion == "Titular").ToList();

    if (titulares.Count == 0)
    {
        MessageBox.Show("No hay jugadores titulares registrados para mostrar.", "Aviso", MessageBoxButtons.OK, MessageBoxIcon.Information);
        return;
    }

    FormCancha ventanaCancha = new FormCancha(titulares);
    ventanaCancha.Show(); //Se muestra el formulario Cancha
}
```



Figura 8

Función dibujar a los jugadores por posición

```
4 referencias
private void DibujarPosicion(Graphics g, string posicionBuscar, Brush colorPunto, float posicionY)
{
    //Filtramos a los jugadores por la posición solicitada
    var jugadoresEnLinea = titulares.Where(j => j.Posicion == posicionBuscar).ToList();

    if (jugadoresEnLinea.Count == 0) return; |

    //Calculamos el espaciado para que se centren automáticamente
    float anchoPantalla = this.ClientSize.Width;
    float espaciado = anchoPantalla / (jugadoresEnLinea.Count + 1);

    int diametro = 30; // Tamaño del punto
    Font fuenteTexto = new Font("Segoe UI", 8, FontStyle.Bold); //fuente

    for (int i = 0; i < jugadoresEnLinea.Count; i++)
    {
        //Calculamos la posición X exacta
        float x = espaciado * (i + 1) - (diametro / 2f);

        //Dibujamos el punto (color según la posición)
        g.FillEllipse(colorPunto, x, posicionY, diametro, diametro);

        //Le ponemos un borde blanco al punto para que resalte
        g.DrawEllipse(new Pen(Color.White, 2), x, posicionY, diametro, diametro);

        //Armamos el texto y lo centramos debajo del punto
        string texto = $"{jugadoresEnLinea[i].Nombre} ({jugadoresEnLinea[i].NumeroCamiseta})";
        SizeF tamañoTexto = g.MeasureString(texto, fuenteTexto);

        float textoX = x + (diametro / 2f) - (tamañoTexto.Width / 2f);
        float textoY = posicionY + diametro + 5;

        g.DrawString(texto, fuenteTexto, Brushes.White, textoX, textoY);
    }
}
```

```
2 referencias
private void FormCancha_Paint(object sender, PaintEventArgs e)
{
    Graphics g = e.Graphics;
    g.SmoothingMode = System.Drawing.Drawing2D.SmoothingMode.AntiAlias;

    DibujarPosicion(g, "Delantero", Brushes.Red, this.ClientSize.Height * 0.15f);
    DibujarPosicion(g, "Mediocampista", Brushes.Yellow, this.ClientSize.Height * 0.40f);
    DibujarPosicion(g, "Defensa", Brushes.Green, this.ClientSize.Height * 0.65f);
    DibujarPosicion(g, "Portero", Brushes.Blue, this.ClientSize.Height * 0.85f);
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

Al ejecutar la aplicación, se observó que la sincronización entre los controles del formulario y la lista en memoria es inmediata. Las validaciones lógicas actúan con precisión: al intentar agregar o modificar un jugador para que sea el "Titular" número 12, el sistema intercepta la acción y protege la regla de negocio correctamente. Por otro lado, la renderización de la cancha demostró ser altamente escalable; independientemente de si hay 3, 4 o 5 defensas registrados, el algoritmo de distribución matemática en el evento Paint reparte el espacio horizontal (`ClientSize.Width`) de forma equitativa, dibujando las posiciones de manera centrada, sin superposiciones (gracias al ajuste de resolución) y con bordes suavizados por el `SmoothingMode.AntiAlias`.

V. CONCLUSIONES

- Se logró desarrollar una aplicación de escritorio 100% funcional que respeta el paradigma de la Programación Orientada a Objetos para gestionar datos complejos en memoria volátil.
- Se identificó la importancia y eficiencia de usar expresiones LINQ combinadas con listas genéricas (`List<T>`) para realizar conteos y filtrados rápidos, lo cual es fundamental para validar reglas de negocio en tiempo real.
- Se demostró que es posible crear interfaces de visualización de datos potentes y dinámicas utilizando GDI+ (`System.Drawing`), transformando simples registros de texto de una tabla en una representación táctica visual (puntos en una cancha) de manera automatizada.

VI. RECOMENDACIONES



INFORME DE LABORATORIO– Alineacion de futbol

- Implementar un sistema de persistencia de datos mediante serialización (por ejemplo, guardar y cargar la lista de jugadores en archivos `.json` o `.xml`), para que la alineación no se pierda al cerrar el programa, respetando la regla de no usar motores de bases de datos relacionales.